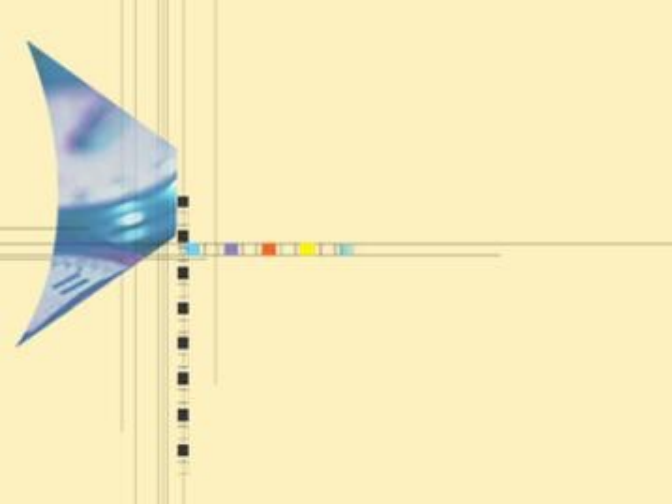




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

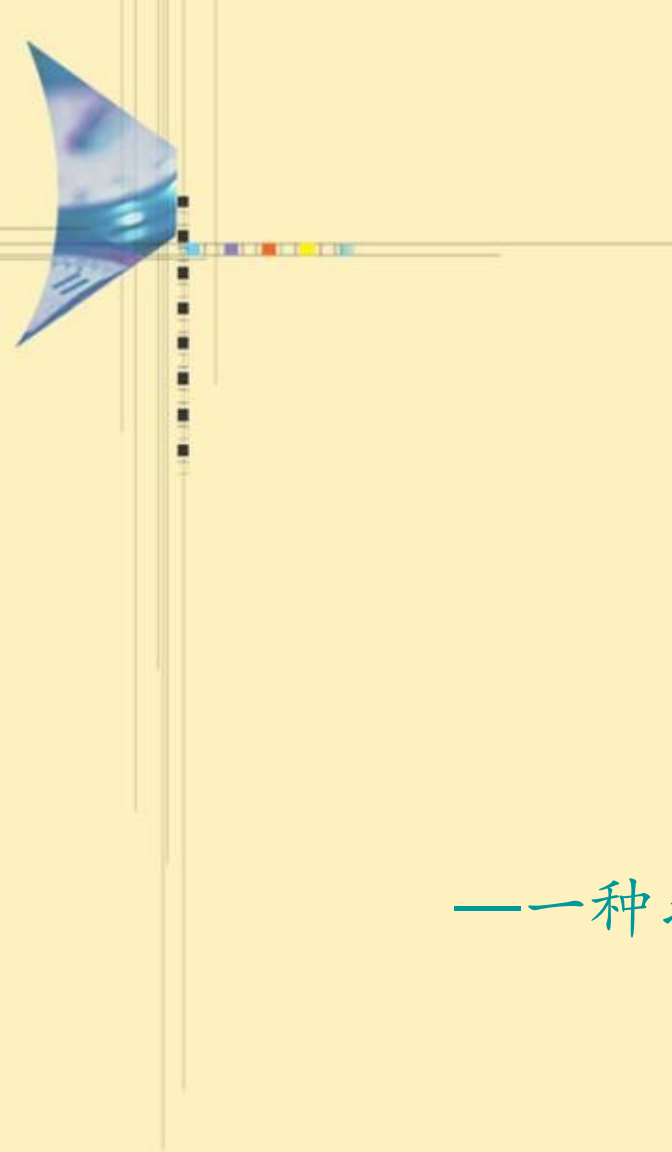
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/1

武汉大学计算机学院

武汉大学
Wuhan University

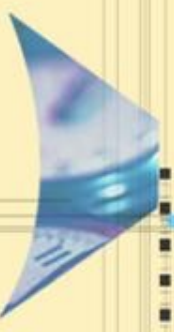




第五讲

贪心算法

——一种求解最优化问题的有效算法



主要内容

5.1 引言

5.2 贪心算法简单例

5.3 贪心法设计思想

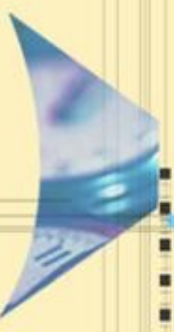
5.4 贪心法正确性证明

5.5 对贪心法得不到最优解情况的处理

5.6 贪心策略的典型应用

5.1 引言

- 当寻找一个问题的解时，也许最先想到的策略就是贪心算法。
- 贪心策略和图遍历的一个共同的特征是局部决策，因此速度快。
- 图遍历技术中，要检查的下一个节点的选择范围被限制在当前节点的一个邻点集中。同检查较大范围的领域相比，则显然是一个简单的贪心策略，因此图遍历算法也可以看作是一个贪心算法。
- 由于是局部决策，因此对有些问题，贪心策略倾向于用来求初始解，很少能提供最优解。因此，其贡献是提供了求解初始解的一种方法，即为仔细考察问题特性设置了一个阶段。



5.1 引言

- 顾名思义，贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的局部最优选择。
- 虽然贪心算法不能对所有问题都得到全局最优解，但对许多问题它的确能产生全局最优解。如单源最短路径问题，最小生成树问题等。
- 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却也是最优解的很好近似。

贪心策略和动态规划策略异同点

相同点：

- 贪心算法和动态规划算法的一样，通常用于求解**最优化问题**，即量的最大化或最小化。

不同点：

- 然而，贪心算法不像动态规划算法，它通常包含一个用以**寻找局部最优解**的迭代过程。在某些实例中，这些局部最优解转变成了全局最优解，而在另外一些情况下，则无法找到全局最优解。而动态规划算法是进行全局优化，得到的是全局最优解。

贪心算法的适用性

- 对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。
- 但是，从许多可用贪心算法求解的问题中我们看到能用贪心算法得到最优解的问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性质**。

贪心选择性质

- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这意味着每一步的最优选择不会影响未来的选择，即当前的选择只依赖于当前的状态，而不依赖于未来的状态
- 这是贪心算法与动态规划算法的主要区别。动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。
- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

最优子结构性质

- 当一个问题最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。这意味着，如果能够将问题分解成若干个子问题，并且每个子问题的最优解能够组合成整个问题的最优解。
- 问题的最优子结构性质是该问题可用动态规划算法或贪心算法求得最优解的关键特征。

可行解、部分可行解、贪心解与最优解

对于一个约束优化问题：

- 如果决策序列是关于一个问题的完整解的形式，且不破坏问题的约束条件，则称它为该问题的**可行解**。
- 如果一个决策序列是问题的完整解形式的一部分，且满足约束条件，则称它为**部分可行解**。
- 不满足问题约束条件的完整或部分解形式的决策序列，称为**不可行的（部分）解**。
- 使目标函数值取最优值的可行解，称为**最优解**。
- 通过贪心算法得到的可行解，称为**贪心解**。
- 贪心解不一定是最优解。



主要内容

5.1 引言

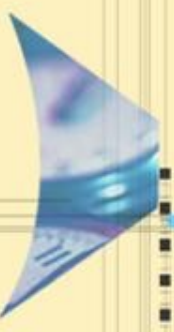
5.2 贪心算法简单例

5.3 贪心法设计思想

5.4 贪心法正确性证明

5.5 对贪心法得不到最优解情况的处理

5.6 贪心策略的典型应用



主要内容

5.2 贪心算法简单例

5.2.1 最短路径问题

5.2.2 最小耗费生成树问题

5.2.1 最短路径问题

问题描述:

- 设 $G = (V, E)$ 是一个每条边有非负长度的有向图，有一个特异顶点 s 称为源。
- 单源最短路径问题，或者简称为最短路径问题，是要确定从 s 到 V 中每一个其他顶点的距离，这里从顶点 s 到顶点 x 的距离定义为从 s 到 x 的最短路径的长度。

Dijkstra算法基本思想

- 为简便起见, 假设 $V = \{1, 2, \dots, n\}$, $s = 1$.
- 初始时, 将顶点分为两个集合 $X = \{1\}$ 和 $Y = \{2, 3, \dots, n\}$ 。这样做的目的是让 X 包含这样的顶点集合: 从源点到这些顶点的距离已经确定。
- 在每一步中, 我们选定源点到它的距离已经获得的一个顶点 $y \in Y$, 并将这个顶点移到 X 中。
- 与 Y 中的每个顶点 y 联系的是标记 $\lambda[y]$, 它是只经过 X 中顶点的最短路径的长度, 一旦顶点 $y \in Y$ 移到 X 中, 与 y 相邻的每个顶点 $w \in Y$ 的标记就被更新, 这表示找到了经过 y 到 w 的更短路径。

算法5.1 DIJKSTRA

输入: 含权有向图 $G=(V,E)$, $V=\{1,2,\dots,n\}$.

输出: G 中顶点 1 到其他顶点的距离。

```
1.  $X=\{1\}$ ;  $Y \leftarrow V-\{1\}$ ;  $\lambda[1] \leftarrow 0$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.     if  $y$  相邻于 1 then  $\lambda[y] \leftarrow \text{length}[1,y]$ 
4.     else  $\lambda[y] \leftarrow \infty$ 
5.     end if
6. end for
7. for  $j \leftarrow 1$  to  $n-1$ 
8.     令  $y \in Y$  使得  $\lambda[y]$  最小
9.      $X \leftarrow X \cup \{y\}$            {将顶点  $y$  加入  $X$ }
10.     $Y \leftarrow Y - \{y\}$          {将顶点  $y$  从  $Y$  中删除}
11.    for 每条边  $(y,w)$ 
12.        if  $w \in Y$  and  $\lambda[y] + \text{length}[y,w] < \lambda[w]$  then
13.             $\lambda[w] \leftarrow \lambda[y] + \text{length}[y,w]$ 
14.        end for
15. end for
```

第1步: $\Theta(n)$ 时间.

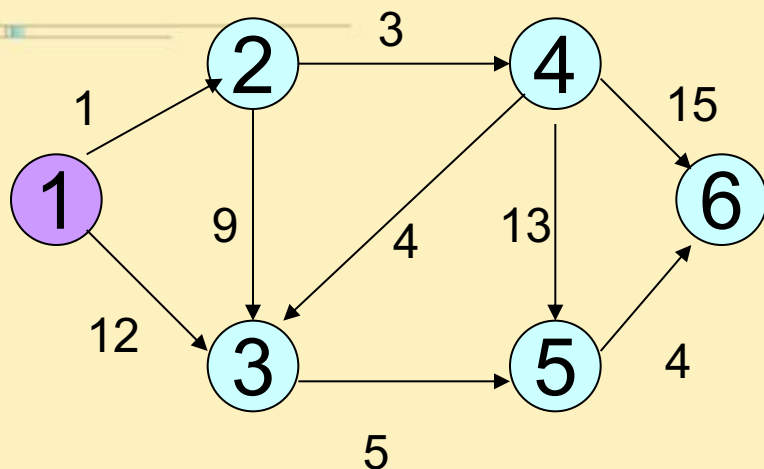
第3步 和第4步: 分别为 $\Theta(n)$ 和 $O(n)$

第8步: $\Theta(n^2)$.

第9步和第10步: 每次迭代花费时间 $\Theta(1)$, 共用了 $\Theta(n)$ 时间.

第11步和第12步共需要时间 $\Theta(m)$. 这里 $m=|E|$.

实例



X

{1}

{1,2}

{1,2,4}

{1,2,4,3}

{1,2,4,3,5}

{1,2,4,3,5,6}

Success!

Y

2	3	4	5	6
1	12	∞	∞	∞

3	4	5	6
10	4	∞	∞

3	5	6
8	17	19

5	6
13	19

6
17

Dijkstra算法正确性证明

引理: 在算法DIJKSTRA中, 当顶点 y 在第8步中被选中, 如果标记 $\lambda[y]$ 是有限的, 那么 $\lambda[y] = \delta[y]$ 。 ($\delta[y]$ 表示从源点 x 到 y 的距离)

证明: 对顶点离开集合 Y 并进入 X 的次序做归纳 (步骤)。

初始: 第1个离开的顶点是1, 我们有 $\lambda[1] = \delta[1] = 0$ 成立。

假设: 假设引理中的结论对于所有在 y 前离开 Y 的顶点都成立。

归纳证明对 y 也成立:

- 因为 $\lambda[y]$ 是有限的, 则必存在从1到 y 的路径, 它的长是 $\lambda[y]$ 。
- 设 $\pi = 1, \dots, x, w, \dots, y$ 是从1到 y 的最短路径, 其中 x 是在 y 前最迟离开 Y 的顶点。
- $\lambda[y] \leq \lambda[w] \leq \lambda[x] + \text{length}(x, w) = \delta[x] + \text{length}(x, w) = \delta[w] \leq \delta[y]$ 。
- 得证。

注: 上述的证明基于这样一个假设, 即所有的边长都是非负的。

Dijkstra算法分析

- 第1步: $\Theta(n)$ 时间.
- 第3步 和第4步: 分别为 $\Theta(n)$ 和 $O(n)$
- 第8步: $\Theta(n^2)$.
- 第9步和第 10步: 每次迭代花费时间 $\Theta(1)$, 共用了 $\Theta(n)$ 时间.
- 第11步和第12步共需要时间 $\Theta(m)$. 这里 $m=|E|$.
- 根据以上得出算法的时间复杂性是 $\Theta(m+n^2) = \Theta(n^2)$.

定理 给出一个边具有非负权的有向图G和源点s , 算法 DIJKSTRA 在时间 $\Theta(n^2)$ 内找出从s到其他每一顶点距离的长度

Floyd与Dijkstra算法的比较

- Dijkstra算法中,图所有边权值都为非负的; Floyd算法中,不允许所有权值为负的回路,
- Dijkstra只可以求出单一源点到其他点的最短距离; Floyd可以求出任意两点间的最短距离;
- Dijkstra算法的思想是贪心, Floyd算法的思想是动态规划;
- Dijkstra时间复杂度 $O(n^2)$, Floyd算法 $O(n^3)$;
- Dijkstra 算法在网络中用得更多, 一个一个节点添加, 加一个点刷一次路由表; Floyd 算法把所有已经连接的路径都标出来, 再通过不等式比较来更改路径。

Dijkstra算法改进

基本思想:

- 用最小堆数据结构来保持集合Y中的顶点，使Y组中离V-Y最近的顶点y可以在 $O(\log n)$ 时间内被选出
- 与每个顶点v相关的键就是它的标记 $\lambda[v]$.
- 最后的算法如算法SHORTESTPATH所示

算法5.2 SHORTESTPATH

输入:含权有向图 $G=(V,E)$, $V=\{1,2,\dots,n\}$.

输出: G 中顶点1到其他顶点的距离。假设已有一个空堆 H 。

1. $Y \leftarrow V - \{1\}$; $\lambda[1] \leftarrow 0$; $\text{key}(1) \leftarrow \lambda[1]$

2. for $y \leftarrow 2$ to n

3. if y 邻接于1 then

4. $\lambda[y] = \text{length}[1,y]$

5. $\text{key}(y) \leftarrow \lambda[y]$

6. $\text{INSERT}(H,y)$

7. else

8. $\lambda[y] \leftarrow \infty$

9. $\text{key}[y] \leftarrow \lambda[y]$

10. end if

11. end for

NEXT PAGE

算法5.2 SHORTESTPATH

```
12. for  $j \leftarrow 1$  to  $n-1$ 
13.      $y \leftarrow \text{DELETEMIN}(H)$ 
14.      $Y \leftarrow Y - \{y\}$     {将顶点 $y$ 从 $Y$ 删除}
15.     for 每个邻接于 $y$ 的顶点  $w \in Y$ 
16.         if  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then
17.              $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$ 
18.              $\text{key}(w) \leftarrow \lambda[w]$ 
19.         end if
20.         if  $w \notin H$  then  $\text{INSERT}(H, w)$ 
21.         else  $\text{SIFTUP}(H, H^{-1}(w))$  { $H^{-1}(w)$ 返回 $w$ 在 $H$ 中的位置}
22.         end if
23.     end for
24. end for
```

$n-1$ 个 **DELETEMIN** 运算,
 $n-1$ 个 **INSERT** 运算,
最多 $m-n+1$ 个 **SHIFTUP** 运算。
每个堆运算用 $O(\log n)$ 时间,
总共需要 $O(m \log n)$ 的时间。

SHORTESTPATH算法分析

- 运行时间主要取决于堆运算。
- 共有 $n-1$ 个DELETEMIN运算， $n-1$ 个INSERT运算，最多 $m-n+1$ 个SHIFTUP运算。每个堆运算用 $O(\log n)$ 时间，得到总共需要 $O(m \log n)$ 的时间。

定理 给出具有非负权重的边和源顶点 s 的图 G ，算法SHORTESTPATH可在 $O(m \log n)$ 时间内找出从 s 到其他每一个顶点的距离。如果图是稠密的，即对于某个 $\epsilon > 0$ ， $m \geq n^{1+\epsilon}$ ，那么它可以被改善为在时间 $O(m/\epsilon)$ 内运行。

需要强调的: 输入算法的是图的邻接表，如果用邻接矩阵，我们最终会得到一个 $\Theta(n^2)$ 时间的算法。

5.2.2 最小耗费生成树

定义5.1:

- 设 $G = (V, E)$ 是一个具有含权边的连通无向图。 G 的一棵生成树 (V, T) 是 G 的作为树的子图。
- 如给 G 加权并且 T 的各边的权的和为最小值, 那么 (V, T) 就称为最小耗费生成树或简称为最小生成树

问题描述

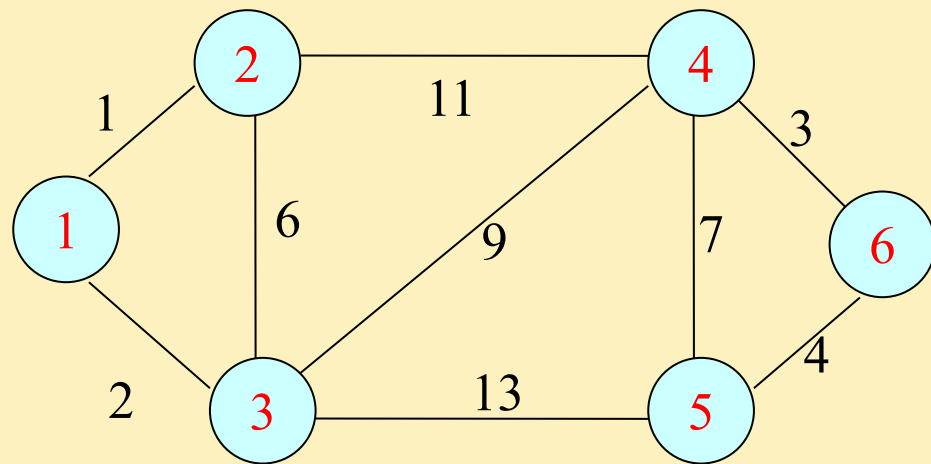
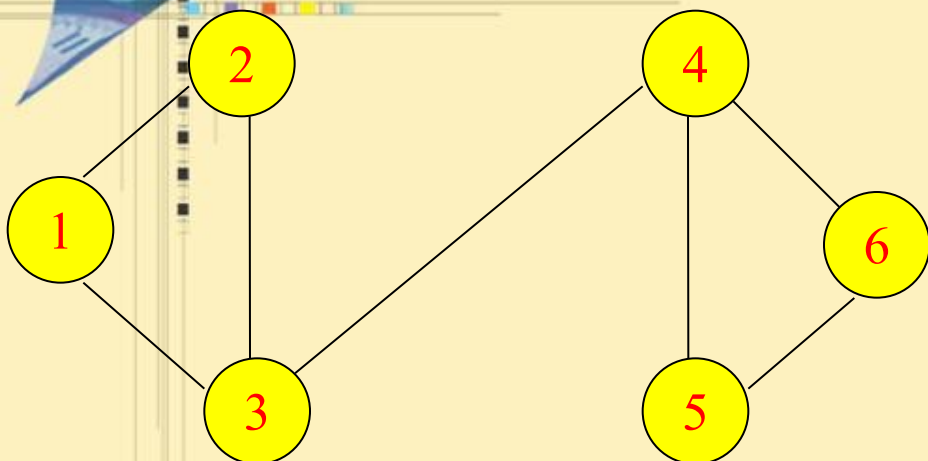
- 假设 G 为含权边的连通无向图, 怎么找到最小生成树(MST)?

Kruskal算法基本思想

工作原理：维护由许多生成树组成的一个森林，这些生成树逐步合并直到最终森林只由一棵树组成，这棵树就是最小耗费生成树。

- 此算法先从对权以非降序排列着手。
- 接着从由图的顶点组成而不包含边的森林 (V, T) 开始，重复下面步骤，直到 (V, T) 被变换成一棵树：
 - 设 (V, T) 是到现在为止构建的森林， $e \in E - T$ 为当前考虑的边，如果把 e 加到 T 中不生成一个回路，那么将 e 加入进 T ，否则丢弃。
 - 这个处理在恰好加完 $n-1$ 条边后结束。

实例



(1,2)	(1,3)	(4,6)	(5,6)	(2,3)	(4,5)	(3,4)	(2,4)	(3,5)
1	2	3	4	6	7	9	11	13

!构成回路, 这条边被丢弃.

Success!

Kruskal算法的实现

- 为有效地实现此算法，我们需要某种机制来检测加入边后**是否构成回路**。需要确定一种数据结构，让它在算法的每个时刻来表示森林，并且在向T中添加边时动态检测是否有回路生成。
- 数据结构：**不相交集，表示由树组成的森林**
 - 开始时，图的每个顶点由一棵包含一个顶点的树表示
 - 在算法的执行过程中，森林中的每个连通分量由一棵树来表示。

算法5.3 KRUSKAL

输入:包含 n 个顶点的含权连通无向图 $G=(V,E)$ 。

输出:由 G 生成的最小耗费生成树 T 组成的边的集合。

1. 按非降序权重将 E 中的边排序
2. **for** 每条边 $v \in V$
3. **MAKESET** $\{v\}$
4. **end for**
5. $T = \{\}$
6. **while** $|T| < n-1$
7. 令 (x,y) 为 E 中的下一条边.
8. **if** $\text{FIND}(x) \neq \text{FIND}(y)$ **then** //检查 x , y 是否在同一颗树中
9. 将 (x,y) 加入 T
10. **UNION** (x,y)
11. **end if**
12. **end while**

➤第1步和第2步分别花费 $O(m \log m)$ 和 $\Theta(n)$,

➤第7步最多执行 m 次,总共要 $O(n)$ 时间。

➤第9步花费 $\Theta(1)$, 再加上第9步执行 $n-1$ 次, 它的总花费是 $O(n)$ 。

➤合并**UNION**运算执行 $n-1$ 次, 查找**FIND**运算最多 $2m$ 次。这两个运算的总花费是 $O(m \log^* n)$ 。

➤算法总时间由第1步排序决定。

Kruskal算法分析

- 第1步和第2步分别花费 $O(m \log m)$ 和 $\Theta(n)$, 这里 $m = |E|$.
- 第7步最多 m 次, 总共要 $O(m)$ 时间。
- 第9步花费 $\Theta(1)$, 再加上第9步最 $n-1$ 次, 它的总花费是 $O(n)$ 。
- 合并运算执行 $n-1$ 次, 查找运算最多 $2m$ 次, 这两个运算的总花费是 $O(m \log^* n)$ (第3章)。
- 这样算法总的运行时间取决于排序步, 也就是 $O(m \log m)$

Kruskal算法正确性证明

引理: 在含权连通无向图中，算法KRUSKAL正确地找出最小生成树。

证明: 对 T 的大小实施归纳， T 是最小生成树的边集。

初始: $T=\{\}$ ，命题平凡为真；

假设: 假设算法第9步加入边 $e=(x,y)$ 之前命题为真，即 $T \subset T^*$ ， T^* 是最小耗费生成树 $G^*=(V, T^*)$ 的边集。

归纳步骤: 设 X 是包含 x 的子树的顶点集，设 $T'=T \cup \{e\}$ ，下面将证明 T' 也是最小耗费生成树中边集的子集。

Kruskal算法正确性证明

根据归纳假设, $T \subset T^*$ 。如果 T^* 包含 e , 那么就不需要进一步证明。假设 T^* 不包含 e , 则 $T^* \cup \{e\}$ 恰好包含以 e 为一条边的一条回路。因为 $e=(x,y)$ 连接了 X 中的一个顶点和 $V-X$ 中的另一个顶点, T^* 必定也包含另一条边 $e'=(w,z)$, 使得 $w \in X$, 并且 $z \in V-X$ 。

根据贪心策略可知, 一定有 $\text{cost}(e') \geq \text{cost}(e)$, 否则 e' 应该在 e 前面添加, 因为它不与 T^* (含有 T 中所有边) 的边构成回路。

构造 $T^{**} = T^* - \{e'\} \cup \{e\}$, 则有 $\text{cost}(T^{**}) \leq \text{cost}(T^*)$, 即 T^{**} 也是最小耗费生成树的边集。此时, $T' \subset T^{**}$, 因此 T' 也是最小耗费生成树中边集的子集。

Prim算法

基本思想

- 算法与求解最短路径的Dijkstra算法相似。从任意顶点开始生长生成树。
- 设 $G=(V,E)$ ，为了简便起见， V 取整数集合 $\{1,2,\dots,n\}$ 。
- 算法从建立两个顶点集合开始: $X=\{1\}$ 和 $Y=\{2,\dots,n\}$ 。接着生长一棵生成树，每次一条边。
- 在每一步，它找出一条权重最小的边 (x,y) ，这里 $x\in X$ ， $y\in Y$ ，并且把 y 从 Y 移到 X 。重复这一步直到 Y 为空。

Prim算法框架

1. $T \leftarrow \{\}$; $X \leftarrow \{1\}$; $Y \leftarrow V - \{1\}$

2. **while** $Y \neq \{\}$

3. 设 (x,y) 是最小权重的边, 其中 $x \in X$ and $y \in Y$

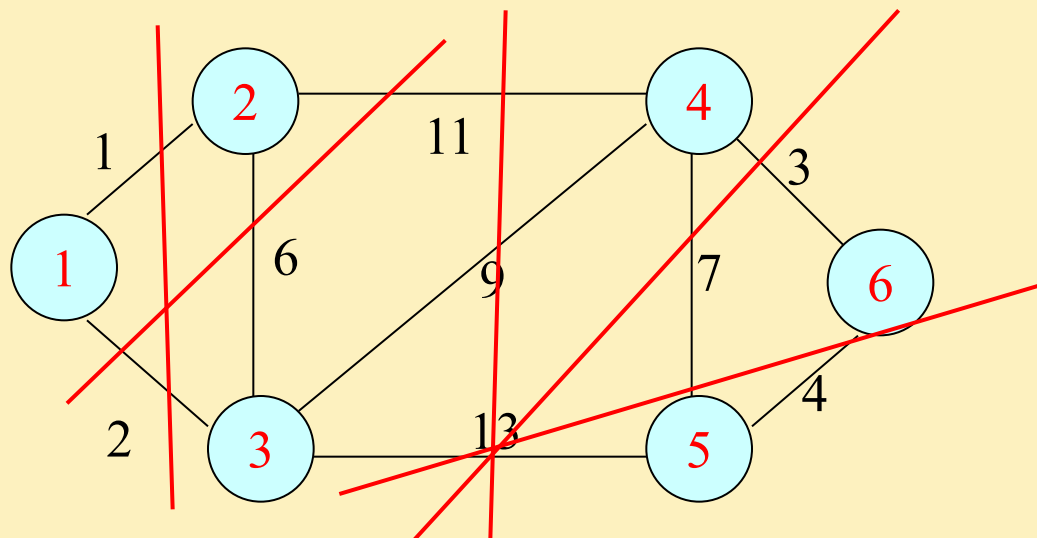
4. $X \leftarrow X \cup \{y\}$

5. $Y \leftarrow Y - \{y\}$

6. $T \leftarrow T \cup \{(x,y)\}$

7. **end while**

实例



Success!

Prim算法实现

算法5.4 PRIM

输入:含权连通无向图 $G=(V,E)$, 其中 $V=\{1,2,\dots,n\}$.

输出:由 G 生成的最小耗费生成树 T 组成的边的集合。

```
1.  $T \leftarrow \{\}$ ;  $X \leftarrow \{1\}$ ;  $Y \leftarrow V - \{1\}$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.   if  $y$  邻接于 1 then
4.      $N[y] \leftarrow 1$ 
5.      $C[y] \leftarrow c[1,y]$ 
6.   else  $C[y] \leftarrow \infty$ 
7.   end if
8. end for
```

➤ 第1步耗费 $\Theta(n)$ 时间.

➤ 第2-8步的for循环需要时间 $\Theta(n)$.

NEXT PAGE

Prim算法实现

算法5.4 PRIM

```
9. for j ← 1 to n-1 //寻找n-1 条边
10.   令 y ∈ Y 使得 C[y] 最小
11.   T ← T ∪ {y, N[y]} //将边 (y, N[y]) 加入 T
12.   X ← X ∪ {y}      // 将顶点 y 加入 X
13.   Y ← Y - {y}      //从Y删除顶点
14.   for 每个邻接于 y 的顶点 w ∈ Y
15.     if c[y, w] < C[w] then
16.       N[w] ← y
17.       C[w] ← c[y, w]
18.     end if
19.   end for
20. end for
```

➤ 第10步搜索离X最近的顶点y，每迭代一次需要花费时间 $\Theta(n)$ ，而这一步执行了n-1次，所以第10步一共花费时间 $\Theta(n^2)$

➤ 第11步、第12步和第13步，每迭代一次花费时间 $\Theta(1)$ 总共花费时间 $\Theta(n)$ 。

➤ 第14步的for循环执行了 $2m$ 次，因为每条 (y, w) 被检查两次。第14步一共需要的时间是 $\Theta(m)$

Prim算法分析

- 第1步耗费 $\Theta(n)$ 时间;
- 第2步的for循环需要时间 $\Theta(n)$;
- 第10步搜索离X最近的顶点y, 每迭代一次需要花费时间 $\Theta(n)$, 而这一步执行了n-1次, 所以第10步一共花费时间 $\Theta(n^2)$;
- 第11步、第12步和第13步, 每迭代一次花费时间 $\Theta(1)$, 总共花费时间 $\Theta(n)$;
- 第14步的for循环执行了2m次, 第14步一共需要的时间是 $\Theta(m)$ 。
- 因此, 算法的时间复杂性是 $\Theta(m+n^2) = \Theta(n^2)$

Prim算法正确性证明

引理: 在含权连通无向图中，算法PRIM正确地找出最小生成树。

证明: 对T的大小实施归纳， (X,T) 是最小生成树的子树。

初始: $T=\{\}$ ，命题平凡为真；

假设: 假设算法第11步加入边 $e=(x,y)$ 之前命题为真，其中 $x \in X, y \in Y$ 。

归纳步骤: 设 $X'=X \cup \{y\}$ ， $T'=T \cup \{e\}$ ，要证明 $G'=(X',T')$ 也是最小耗费生成树的子树。

Prim算法正确性证明

首先，证明 G' 是一棵树。由于 e 恰好与 X 中的一个称为 x 的顶点连接，同时，根据归纳假设， (X, T) 是一棵树。有因为 y 不与 X 中，因此， G' 连通且无回路，即是一棵树。

接着，证明 G' 是一棵最小耗费成本生成树的子树。

根据归纳假设， $T \subset T^*$ ，这里 T^* 是最小生成树 $G^*=(V, T^*)$ 的边集。如果 T^* 包含 e ，那么就不需要进一步证明。假设 T^* 不包含 e ，则 $T^* \cup \{e\}$ 恰好包含以 e 为一条边的一条回路。由于 $e=(x,y)$ 连接了 X 中的一个顶点和 Y 中的另一个顶点， T^* 必定也包含另一条边 $e'=(w,z)$ ，使得 $w \in X$ ，并且 $z \in Y$ 。根据贪心策略可知，一定有 $\text{cost}(e') \geq \text{cost}(e)$ ，否则 e' 应该在 e 前面添加，因为它不与 T^* （含有 T 中所有边）的边构成回路。构造 $T^{**}=T^* - \{e'\} \cup \{e\}$ ，则有 $\text{cost}(T^{**}) \leq \text{cost}(T^*)$ ，即 T^{**} 也是最小耗费生成树的边集。此时， $T' \subseteq T^{**}$ ，因此 G' 也是最小耗费生成树的子树。

Prim算法改进

基本思想

- 现在要改进算法PRIM，就像曾经对算法DIJKSTRA所做的那样，目标是把 $m = o(n^2)$ 的那类图的时间复杂性从 $\Theta(n^2)$ 减少到 $O(m \log n)$ 。以后还要进一步改进它，使在稠图情况下，可以在对边数的线性时间内运行。
- 就像在算法SHORTESTPATH中那样，基本思想是用**最小堆**数据结构（第3章）来保持边界顶点集，使得可以在 $O(\log n)$ 时间内取得Y集中的顶点y，这个y和V-Y集中一个顶点连接的边的耗费是最小的。修改后的算法在算法MST中给出。

算法5.5 MST

输入:含权连通无向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

输出:由 G 生成的最小耗费生成树 T 组成的边的集合。假设我们已有一个空堆 H .

1. $T \leftarrow \{\}$; $Y \leftarrow V - \{1\}$
2. **for** $y \leftarrow 2$ **to** n
3. **if** y 邻接于1 **then**
4. $N[y] \leftarrow 1$
5. $\text{key}(y) \leftarrow c[1,y]$
6. $\text{INSERT}(H,y)$
7. **else** $\text{key}(y) \leftarrow \infty$
8. **end if**
9. **end for**

NEXT PAGE

算法5.5 MST

```
10. for  $j \leftarrow 1$  to  $n-1$            {查找 $n-1$  条边}
11.    $y \leftarrow \text{DELETEMIN}(H)$ 
12.    $T \leftarrow T \cup \{(y, N[y])\}$    {添加边  $(y, N[y])$  到  $T$ }
13.    $Y \leftarrow Y - \{y\}$            {从  $Y$  删除顶点  $y$ }
14.   for 每个邻接于  $y$  的顶点  $w \in Y$ 
15.     if  $c[y, w] < \text{key}(w)$  then
16.        $N[w] \leftarrow y$ 
17.        $\text{key}(w) \leftarrow c[y, w]$ 
18.     end if
19.     if  $w \notin H$  then  $\text{INSERT}(H, w)$ 
20.     else  $\text{SIFTUP}(H, H^{-1}(w))$ 
21.   end for
22. end for
```

MST算法时间复杂性

定理 给出一个含权无向图 G ，算法MST在 $O(m \log n)$ 时间内找出最小耗费生成树。如果图是稠密的，即如果对于某个 $\varepsilon > 0$, $m \geq n^{1+\varepsilon}$ ，那么它可以被进一步改进为在 $O(m/\varepsilon)$ 时间内运行。

